

CSI247: Lab 1

August 11, 2022

Lab 1: Objects and Inheritance

In this lab, you will get a taste of how objects and inheritance works in Java. At the end of the lab you should be able to:

- Create an class.
- Inherit from the class
- Overriding super class method.
- Follow proper java project folder structure and compile from the command line.

Project Structure

Follow the steps below to create appropriate project structure for this lab.

- In your *Z* drive, create a directory called *CSI247*.
- Inside *CSI247*, create a directory called *lab1*.
- Add the *src* and *classes* to *lab1* directory.
- Create the directory structure for the package *lab1.employee* in the *src* directory.
- Open Visual Studio Code
- In VS Code, go to *File -> Open Folder* and select the *lab1* folder you created.

You should end up with the structure in Figure 1, but without the Java files.

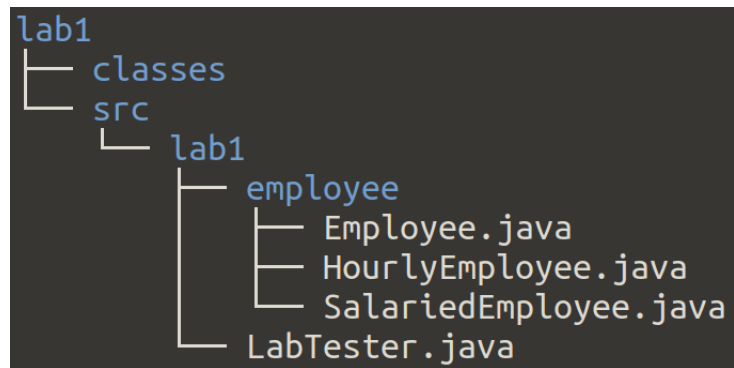


Figure 1: The project directory structure for the lab. You can ignore the java files as you will add them in the coming sections.

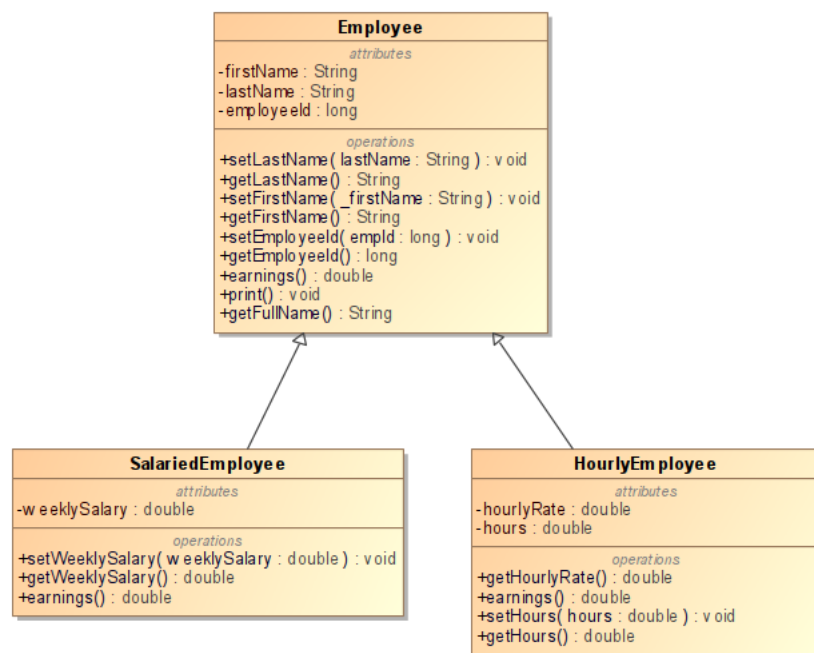


Figure 2: A UML diagram of the class hierarchy.

Lab 1 Classes

In this lab you are going to implement the class hierarchy given in Figure 2. All the three classes should be in the *lab1.employee* package. Add the necessary package declaration to the java classes.

Employee Class

Create the class *Employee* in the package "lab1.employee". Add all the instance variables and methods.

- Create a constructor that takes three parameters to initialise all the instance variables.
- Implement the getters and setters of the class.
- The *fullName()* method should return a combination of the first and last names separated by space.
- The *earnings()* method should return 0
- The *print()* method should use the *printf()* method to output the details of the employee formatted as you want. Make sure you are also printing the earnings of the employee too.

SalariedEmployee Class

A salaried employee has a fixed weekly salary. Their earnings are calculated monthly. In this lab, you can assume that there are 4 weeks in a month.

- Create a constructor that takes the three parameters for the super class plus two parameters for the class' instance variables. In Java, we can call the constructor from the super class using the keyword *super()*. The first line of the constructor should call the super class constructor with the appropriate parameters.
- Implement the getters and setters of the class.
- The *earnings()* method should return a monthly salary.

HourlyEmployee Class

A hourly employee has a fixed hourly salary. The earnings are calculated monthly using the hours worked weekly. In this lab, you can assume that there are 4 weeks in a month.

- Create a constructor that takes the three parameters for the super class plus one parameter for the class' instance variable. In Java, we can call the constructor from the super class using the keyword *super()*. The first line of the constructor should call the super class constructor with the appropriate parameters.

- Implement the getters and setters of the class. Note that there is no setter for *hourlyRate*. This means that once it is set by the constructor, it can no longer be changed.
- The *earnings()* method should return a monthly salary calculated using the hourly rate, the hours and the number of weeks in a month.

Tester Class

- Create a class called LabTester in the *lab1* package.
- Add the *main* method.
- Declare and create an object for each of the classes.
- Declare two more variables of type Employee. Create an object of type SalariedEmployee and assign it to one of the variables. Create another instance of type HourlyEmployee and assign it to the remaining variable.
- Try assigning an object of type Employee to variables of the subclasses.
- Call the *print* methods for each of the objects.
- You should now be able to compile the project using the command `"javac -d classes -sourcepath src src/lab1/LabTester.java"`
- Run the program using the command `"java -classpath classes lab1.LabTester"`.